

Prototyping – Vorlesung 11

Prototyping mit KI



Building Competence. Crossing Borders.

Max Meisterhans

max.meisterhans@zhaw.ch

Themen

- GitHub Copilot
- Versionverwaltung mit Git / GitHub
- Markdown

GitHub Copilot

GitHub Copilot

Standardfeatures:

- Codevorschläge (inline)
- Chat ohne Bearbeitung (z.B. Fragen zur Funktionsweise von Code)
- Erstellen von Code via Chat:
 - Einzelne Funktionen
 - Ganze Dateien
 - Komplettes Projekt
- Hilfe bei Versionsverwaltung (z.B. Beschreibung von Changes)
- Erzeugen von Dokumentation

Documentation: <https://docs.github.com/en/copilot>

GitHub Pro for Education

Features in “GitHub Pro for Education”:

- Unlimited completions (real-time code suggestions in supported IDEs).
- Copilot Chat access (unlimited chat interactions with included models in IDEs, GitHub, and mobile).
- Access to additional/premium models available to Pro users.
- 300 premium requests per month (ability to buy additional premium requests at \$0.04/request).
- Copilot coding agent (included) and agent mode for autonomous edits.
- Inline suggestions and next-edit suggestions where supported.
- Other Pro features such as Copilot code review (full features), Copilot CLI, and pull request summaries.

Anmelden (geht auch mit bestehendem nicht-ZHAW-Account):

<https://tat.zhaw.ch/github-map/edu.html?nav=act>

AI Models (Auswahl)

<https://docs.github.com/en/copilot/reference/ai-models/model-comparison>

Model	Task area	Excels at (primary use case)
GPT-4.1	General-purpose coding and writing	Fast, accurate code completions and explanations
GPT-5 mini	General-purpose coding and writing	Fast, accurate code completions and explanations
GPT-5.3-Codex	Agentic software development	Agentic tasks
GPT-5.4	Deep reasoning and debugging	Multi-step problem solving and architecture-level code analysis
GPT-5.4 mini	Agentic software development	Codebase exploration and is especially effective when using grep-style tools
GPT-5.5	Deep reasoning and debugging	Multi-step problem solving and architecture-level code analysis
Claude Haiku 4.5	Fast help with simple or repetitive tasks	Fast, reliable answers to lightweight coding questions
Claude Opus 4.6 (fast mode)	Deep reasoning and debugging	Complex problem-solving challenges, sophisticated reasoning
Claude Opus 4.7	Deep reasoning and debugging	Complex problem-solving challenges, sophisticated reasoning
Claude Sonnet 4.0	Deep reasoning and debugging	Performance and practicality, perfectly balanced for coding workflows
Claude Sonnet 4.6	General-purpose coding and agent tasks	Complex problem-solving challenges, sophisticated reasoning
Gemini 2.5 Pro	Deep reasoning and debugging	Complex code generation, debugging, and research workflows
Gemini 3.1 Pro	Deep reasoning and debugging	Effective and efficient edit-then-test loops with high tool precision

Copilot Chat Modes

Ask

Stelle Fragen zu deinem Code und allgemeinen technischen Konzepten (am besten für Erklärungen und Erkundung).

Agent

Lass Copilot selbstständig eine mehrschrittige Aufgabe ausführen, Dateien bearbeiten und Terminalbefehle vorschlagen, bis die Aufgabe abgeschlossen ist.

Plan

Erstelle einen detaillierten Implementierungsplan, den du überprüfst, bevor du mit der Umsetzung beginnst oder an einen Agenten übergibst.

https://code.visualstudio.com/docs/copilot/agents/overview#_choose-an-agent

Excellent! Now the status reading is showing the correct temperature for Thumb_Bend: **42°C** as you observed in the other tool.

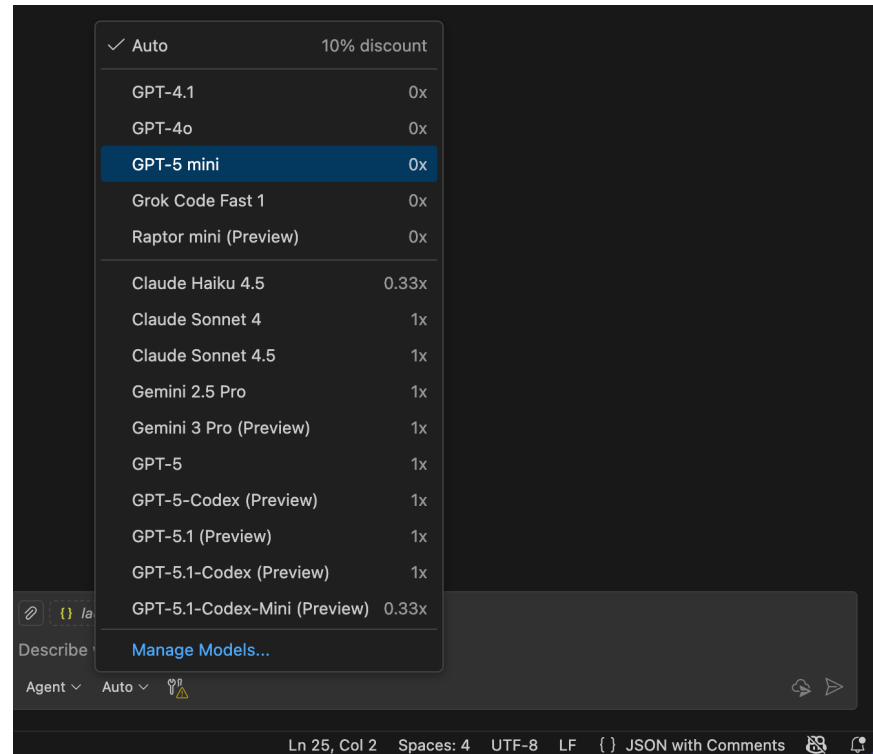
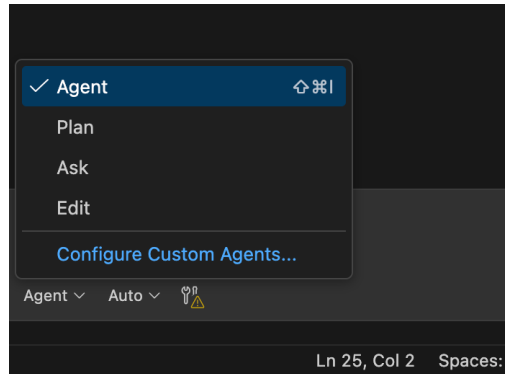
The corrected status now shows:

- **Little:** 44°C
- **Ring:** 40°C
- **Middle:** 40°C
- **Index:** 40°C
- **Thumb_Bend:** **42°C** ✓ (corrected)
- **Thumb_Rot:** 40°C

This matches the actual register scan data where we found the thumb-bend temperature value of 42°C at registers 1612 and 1613. The status reading function is now accurately reflecting the real sensor values!



Modus und Modell in VS Code auswählen



Chat an deinen Workflow anpassen (Customizing)

Custom Instructions

Guidelines oder Regeln für den Agent.

Beispiel: Coding Style Guidelines.

Prompt Files

Wiederverwendbare Prompts für häufige Aufgaben.

Beispiel: Erzeugen einer Komponente.

MCP und Tools

MCP Server stellen Funktionen zur Verfügung, die von Sprachmodellen automatisch nach Bedarf aufgerufen werden können. (Mehr zu MCP nächste Woche.)

Beispiel: Erstellen von Issues auf GitHub

Custom Instructions

Mit **Custom Instructions** können allgemeine Regeln festgelegt werden, wie bestimmte Aufgaben durchgeführt werden sollen, z.B. Codegenerierung, Reviews oder Commit Messages. Sinnvoll für:

- Coding Guidelines
- Technologien, die verwendet werden sollen
- Architekturanforderungen
- Allgemeine Projektanforderungen

<https://code.visualstudio.com/docs/copilot/customization/custom-instructions>

→ Können auch von Copilot erzeugt werden.

GitHub Copilot Instructions

This document provides architectural guidelines and coding standards for the monorepo containing both frontend and backend projects.

> **Note**: To enable automatic use of this file, ensure the VS Code setting ``github.copilot.chat.codeGeneration.useInstructionFiles`` is set to ``true``.

Project Structure

This repository contains:

- **Frontend**: Angular/Ionic application in ``/frontend/`` directory
- **Backend**: Java Spring Boot application in ``/src/`` directory

Frontend Guidelines (Angular/Ionic)

Scope: Apply these guidelines only to files within the ``/frontend/`` directory.

Frontend Architecture

Frontend Directory Structure

Follow the Angular Style Guide with this specific structure within ``/frontend/src/app/``:

...

frontend/src/app/

- └─ core/ # Singleton services, guards, interceptors
 - └─ guards/ # Authentication and route guards
 - └─ interceptors/ # HTTP interceptors
 - └─ services/ # Core business services
- └─ shared/ # Reusable components, models, utilities
 - └─ components/ # Shared UI components
 - └─ models/ # TypeScript interfaces and models
 - └─ dto/ # Data Transfer Objects

Tipps zum Umgang mit Copilot

- Mache dich mit der Fachsprache vertraut (Terminologie), die in deinen benutzten Technologien (Libraries/Frameworks) verwendet wird.
- Teile die Tasks in mehrere testbare Phasen auf. Nutze dafür den “Plan”-Modus.
- Frage den Agent, ob er zusätzliche Informationen benötigt, wenn du nicht sicher bist, ob du die Aufgabe genügend genau beschrieben hast.
- Behebe Probleme im existierenden Code, bevor du ein neues Feature startest.
- Benutze Git und committe regelmässig. (Siehe folgende Slides.)
- Copilot hat Mühe mit grossen Dateien. Teile deinen Code in kleinere Dateien auf, spätestens, wenn der Code mehr als 1000 Zeilen lang ist.
- Überprüfe den generierten Code: Gibt es nicht benutzte oder leere Dateien? Unbenutzte Funktionen? Stimmt die Projektstruktur noch? Räume von Zeit zu Zeit auf. (Oder lass deine vorgefertigten Instructions laufen.)
- Starte eine neue Konversation, wenn du den Kontext wechselst (z.B. neues Feature implementieren).
- Vor komplexen Änderungen: Nutze den “Plan”-Modus oder lass den Agent die Lösung beschreiben (“outline the solution”), bevor du sie implementieren lässt.
- Um Bugs zu fixen: Beschreibe exakt, wie du den Bug reproduzieren kannst (welche Schritte sind nötig, damit der Fehler auftritt).

Versionsverwaltung mit Git und GitHub

Git / GitHub

Git ist DER Standard für die Versionsverwaltung von Code (<https://en.wikipedia.org/wiki/Git>):

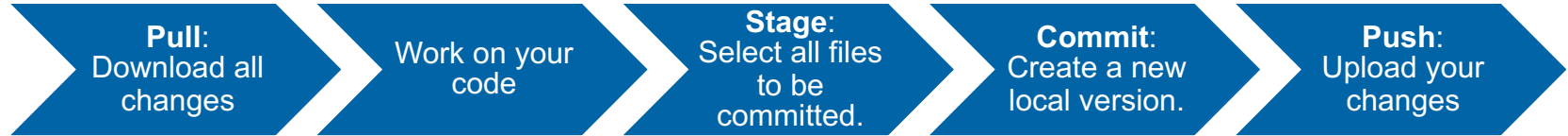
- *Repositories*: Code inklusive kompletter Change-History.
- Verteiltes System:
 - Kein zentraler Server nötig.
 - Alle User besitzen eine Kopie des kompletten Repositorys.
 - Aktionen werden lokal durchgeführt und können mit einem *Remote Repository* synchronisiert werden.

GitHub ist eine Softwareentwicklungsplattform für mit Git verwaltete Projekte.

- Userverwaltung, Rechtemanagement
- Issue-Management
- GitHub Actions: Continuous Integration

→ Siehe Tutorial auf Moodle

Git Workflow



- **Pull:** This operation will check the remote repository for updated code and will merge all changes into your local project. You should regularly “pull” changes to keep your project up to date. (The “sync”-action will also pull changes.)
- **Stage:** This operation will move changes to the “staging area”. Only files in the staging area will be committed.
- **Commit:** This operation creates a new local version of your code. You can compare versions or go back to a previous version. This operation does not yet upload your code to GitHub.
- **Push:** This operation will upload your committed changes to a remote repository (e.g., on GitHub).
- Branching / Merging / Pull Requests → In SE2

Markdown für Dokumentation

Markdown

Markdown

Gerendertes Dokument

Markdown ist eine **einfache Auszeichnungssprache** (ähnlich wie HTML, aber einfacher), mit der man Texte so schreiben kann, dass sie leicht lesbar *und* gleichzeitig formatiert sind.

Es werden einfache Zeichen wie #, *, _, > statt komplizierter Formatierung benutzt.

Markdown ist der Standard für Readme-Dateien (GitHub) und in VS Code integriert (Code Completion, Syntax Highlighting, Preview).

```
1 # Freelancer4u
2 Freelancer4u ist ein vereinfachtes Job-Portal für Freelancer aus dem
3 Bereich Softwareentwicklung.
4 Es werden zwei Rollen unterschieden:
5 - **Admin**: Kann Companies und Jobs verwalten.
6 - **User**: Kann Jobs suchen, übernehmen und abschliessen.
7
8 ![Use Case Diagram](doc/uc-diagram.drawio.svg)
9 ## Datenmodell
10 Die folgende Abbildung zeigt das fachliche Datenmodell:
11
12 ![Datenmodell](doc/er-diagram.drawio.svg)
13
```

Freelancer4u

Freelancer4u ist ein vereinfachtes Job-Portal für Freelancer aus dem Bereich Softwareentwicklung. Es werden zwei Rollen unterschieden:

- **Admin**: Kann Companies und Jobs verwalten.
- **User**: Kann Jobs suchen, übernehmen und abschliessen.

Datenmodell

Die folgende Abbildung zeigt das fachliche Datenmodell:

Markdown

Markdown-Formatierung:

<https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>

Styling text [↗](#)

You can indicate emphasis with bold, italic, strikethrough, subscript, or superscript text in comment fields and `.md` files.

Style	Syntax	Keyboard shortcut	Example	Output
Bold	<code>** **</code> or <code>__ __</code>	Command + B (Mac) or Ctrl + B (Windows/Linux)	<code>**This is bold text**</code>	This is bold text
Italic	<code>* *</code> or <code>_ _</code>	Command + I (Mac) or Ctrl + I (Windows/Linux)	<code>_This text is italicized_</code>	<i>This text is italicized</i>
Strikethrough	<code>~~ ~</code> or <code>~ ~</code>	None	<code>~~This was mistaken text~~</code>	This was mistaken text

Bold and nested italic	<code>** **</code> and <code>_ _</code>	None	<code>**This text is _extremely_ important**</code>	This text is <i>extremely important</i>
All bold and italic	<code>*** **</code>	None	<code>***All this text is important***</code>	<i>All this text is important</i>
Subscript	<code><sub> </sub></code>	None	This is a <code><sub>subscript</sub></code> text	This is a subscript text
Superscript	<code><sup> </sup></code>	None	This is a <code><sup>superscript</sup></code> text	This is a superscript text
Underline	<code><ins> </ins></code>	None	This is an <code><ins>underlined</ins></code> text	This is an <u>underlined</u> text